Data Science sous R

Chapitre 5 — Analyse en composantes principales, arbres de décision et forêts aléatoires

ESME — 2026-2027

Table des matières

5 Analyse en composantes principales, arbres de décision et forêts aléatoires 2
5.1 Analyse en composantes principales (ACP) 2
5.2 Arbres de décision 11
5.3 Forêts aléatoires 16
5.4 Résumé 24
5.5 Exercices 24

Chapitre 5

Analyse en composantes principales, arbres de décision et forêts aléatoires

Objectifs du chapitre

- Comprendre le principe de l'analyse en composantes principales (ACP).
- Réaliser une ACP avec `FactoMineR`, lire le cercle des corrélations et le plan des individus.
- Utiliser des variables illustratives (supplémentaires) quantitatives et qualitatives pour interpréter les axes.
- Construire, élaguer et utiliser un arbre de décision avec `rpart`.
- Évaluer un classifieur : matrice de confusion, sensibilité, spécificité, courbe ROC et AUC.
- Comprendre le principe des forêts aléatoires, pour la classification et la régression, et les mettre en œuvre avec `randomForest`.

```
library(tidyverse)
library(FactoMineR)    # ACP
library(factoextra)    # graphiques des résultats d'ACP
library(rpart)         # arbres de décision
library(pROC)          # courbes ROC
library(randomForest) # forêts aléatoires
library(ISLR)          # jeux de données OJ et Caravan
source("R/boutique.R")
```

5.1 Analyse en composantes principales (ACP)

5.1.1 Le problème

On dispose de n individus décrits par p variables **quantitatives**. Avec $p = 2$, un nuage de points suffit pour tout voir ; avec $p = 10$, c'est impossible. L'ACP cherche à **résumer** le tableau par un petit nombre de nouvelles variables, les **composantes principales**, qui :

- sont des **combinaisons linéaires** des variables d'origine ;
- sont **non corrélées** entre elles ;
- conservent le **maximum de variabilité** (d'« information ») du nuage.

Géométriquement, on cherche le plan sur lequel la projection du nuage de points est la moins déformée possible, comme on choisirait l'angle de vue qui montre le mieux un objet en 3D sur une photo.

L'ACP répond à deux questions :

1. **Individus** : quels individus se ressemblent ? Quels profils se dégagent ?
2. **Variables** : quelles variables sont liées entre elles ? Peut-on résumer plusieurs variables corrélées par une seule dimension ?

En marketing, elle sert par exemple à résumer des dizaines d'indicateurs de comportement client en deux ou trois dimensions interprétables, à construire des **cartes perceptuelles** (positionnement des marques selon les notes données par des consommateurs), ou à préparer une segmentation (chapitre 6).

5.1.2 Principe mathématique

1. **Centrer-réduire** les variables : $z_{ij} = (x_{ij} - \bar{x}_j)/s_j$. C'est l'**ACP normée**, à utiliser dès que les variables ont des unités ou des ordres de grandeur différents (euros, jours, nombres de commandes).
2. Calculer la **matrice des corrélations** R ($p \times p$).
3. Diagonaliser R : ses **vecteurs propres** $u_1, \ldots, u_p$ donnent les directions des axes, ses **valeurs propres** $\lambda_1 \geq \ldots \geq \lambda_p$ la variance portée par chaque axe.
4. Coordonnées des individus sur l'axe k : $F_k = Zu_k$.

En ACP normée, la variance totale (l'**inertie**) vaut p, et la part d'inertie expliquée par l'axe k est λ_k/p.

```
# L'ACP « à la main » sur trois variables de mtcars
Z <- scale(mtcars[, c("mpg", "hp", "wt")])   # centrer-réduire
R <- cor(Z)
decomposition <- eigen(R)
decomposition$values                          # valeurs propres ; leur somme
 ↪  vaut 3
```

```
#> [1] 2.5382 0.3521 0.1097
```

```
decomposition$values / sum(decomposition$values)   # part d'inertie
```

```
#> [1] 0.84606 0.11736 0.03657
```

5.1.3 Exemple 1 : le décathlon

Le jeu `decathlon` (package `FactoMineR`) est l'exemple de référence de l'ACP : les performances de 41 athlètes aux dix épreuves du décathlon, lors de deux compétitions (Décastar et Jeux olympiques de 2004), ainsi que leur classement et leur nombre de points.

```
data(decathlon)
head(decathlon[, 1:6])
```

```
#>          100m Long.jump Shot.put High.jump  400m 110m.hurdle
#> SEBRLE  11.04      7.58    14.83      2.07 49.81       14.69
#> CLAY    10.76      7.40    14.26      1.86 49.37       14.05
#> KARPOV  11.02      7.30    14.77      2.04 48.37       14.09
#> BERNARD 11.02      7.23    14.25      1.92 48.93       14.99
#> YURKOV  11.34      7.09    15.19      2.10 50.42       15.31
#> WARNERS 11.11      7.60    14.31      1.98 48.68       14.23
```

Les dix épreuves sont les variables **actives** : elles construisent les axes. `Rank` et `Points` sont des variables **quantitatives illustratives**, `Competition` une variable **qualitative illustrative** : elles ne participent pas au calcul des axes, mais aident à les interpréter.

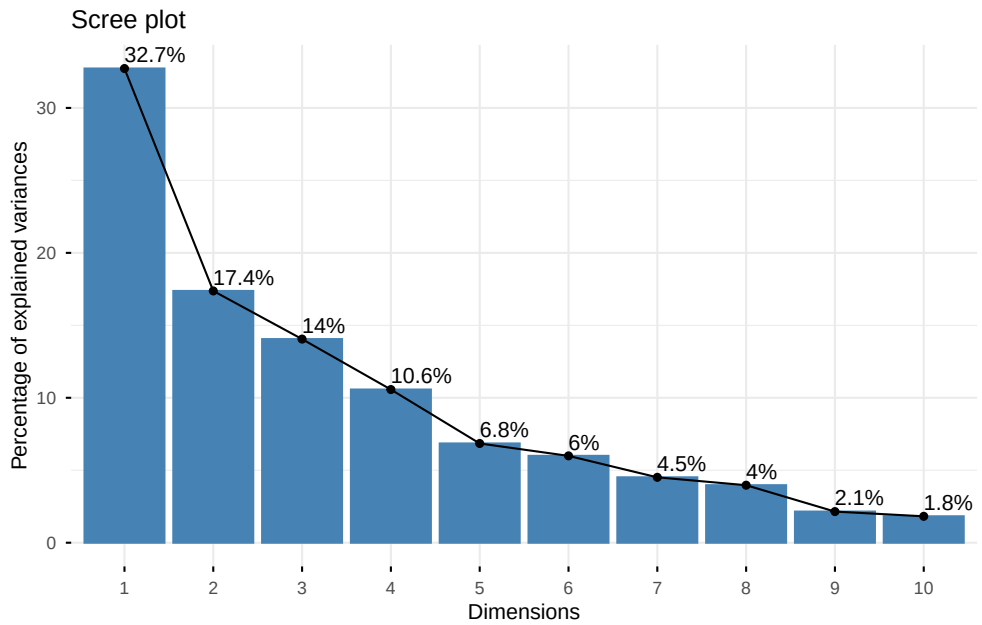
```
res_acp <- PCA(decathlon,
               quanti.sup = 11:12,    # Rank, Points
               quali.sup  = 13,       # Competition
               scale.unit = TRUE,     # ACP normée
               graph = FALSE)
```

5.1.3.1 Combien d'axes retenir ?

```
res_acp$eig[1:5, ]
```

```
#>        eigenvalue percentage of variance
#> comp 1     3.2719                 32.719
#> comp 2     1.7371                 17.371
#> comp 3     1.4049                 14.049
#> comp 4     1.0569                 10.569
#> comp 5     0.6848                  6.848
#>        cumulative percentage of variance
#> comp 1                             32.72
#> comp 2                             50.09
#> comp 3                             64.14
#> comp 4                             74.71
#> comp 5                             81.56
```

```
fviz_eig(res_acp, addlabels = TRUE)
```



Critères usuels (à combiner, pas à appliquer mécaniquement) :

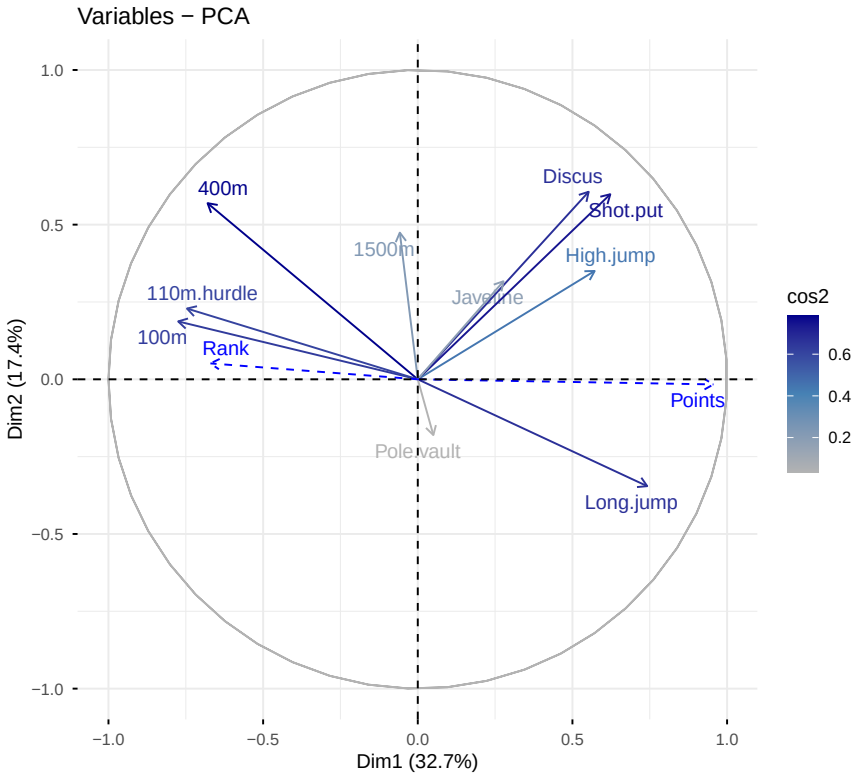
- le **coude** de l'éboulis des valeurs propres ;
- la **règle de Kaiser** (ACP normée) : garder les axes de valeur propre supérieure à 1, c'est-à-dire ceux qui résument plus qu'une variable d'origine ;
- un **pourcentage d'inertie cumulé** suffisant ;
- surtout, l'**interprétabilité** des axes.

Ici, les deux premiers axes expliquent environ 50 % de l'inertie. C'est modeste, ce qui est fréquent quand les variables sont nombreuses et peu corrélées.

5.1.3.2 Le cercle des corrélations

Chaque variable est représentée par une flèche dont les coordonnées sont ses **corrélations** avec les deux axes.

```
fviz_pca_var(res_acp, repel = TRUE, col.var = "cos2",
             gradient.cols = c("grey70", "steelblue", "darkblue"))
```



Règles de lecture

- Une flèche **proche du cercle** : variable bien représentée dans le plan. Une flèche courte est mal représentée ; on ne l'interprète pas.
- Deux flèches **proches** : variables corrélées positivement.
- Deux flèches **opposées** : corrélées négativement.
- Deux flèches **orthogonales** : non corrélées (si elles sont bien représentées).

Interprétation : le premier axe oppose les épreuves de **vitesse** (100 m, 400 m, 110 m haies, dont les temps faibles sont bons) au saut en longueur. Comme un bon sprinteur a un **temps faible**, les variables de course sont à l'opposé du saut en longueur, alors qu'il s'agit des mêmes qualités athlétiques. Le deuxième axe est porté par les **lancers** (poids, disque) mais aussi par le 400 m et le 1 500 m : il distingue les athlètes puissants mais plus lents sur les courses longues (en haut) des athlètes plus légers et endurants (en bas).

La variable illustrative `Points` est très corrélée au premier axe : celui-ci mesure donc la **performance globale**.

```
round(res_acp$var$contrib[, 1:2], 1)     # contribution (%) de chaque variable
 ↪  aux axes
```

```
#>             Dim.1 Dim.2
#> 100m         18.3   2.0
#> Long.jump    16.8   6.9
#> Shot.put     11.8  20.6
```

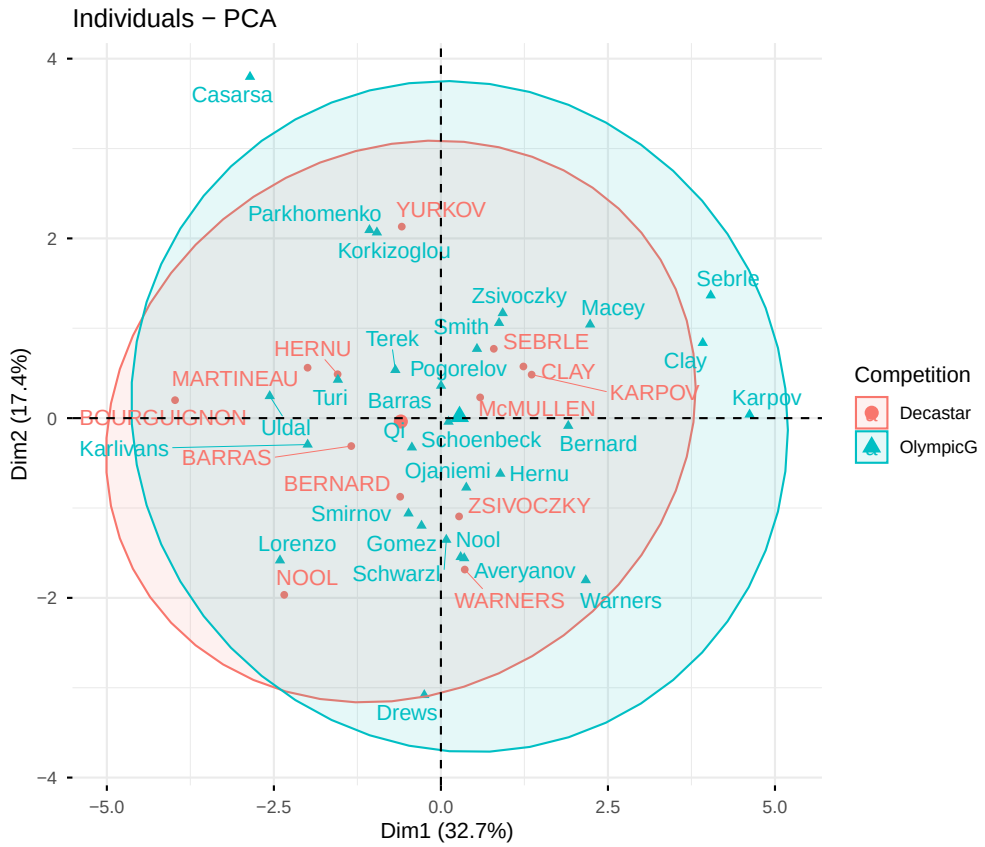
```
#> High.jump    10.0   7.1
#> 400m         14.1  18.7
#> 110m.hurdle  17.0   3.0
#> Discus        9.3  21.2
#> Pole.vault    0.1   1.9
#> Javeline      2.3   5.8
#> 1500m         0.1  12.9
```

```
round(res_acp$var$cos2[, 1:2], 2)        # qualité de représentation
```

```
#>              Dim.1 Dim.2
#> 100m          0.60  0.04
#> Long.jump     0.55  0.12
#> Shot.put      0.39  0.36
#> High.jump     0.33  0.12
#> 400m          0.46  0.32
#> 110m.hurdle   0.56  0.05
#> Discus        0.31  0.37
#> Pole.vault    0.00  0.03
#> Javeline      0.08  0.10
#> 1500m         0.00  0.22
```

5.1.3.3 Projection des individus

```
fviz_pca_ind(res_acp, repel = TRUE,
             habillage = "Competition",    # couleur selon la variable
              ↪  qualitative illustrative
             addEllipses = TRUE)
```



- Deux individus **proches** ont des profils semblables (s'ils sont bien représentés : `cos2` élevé).

- La position d'un individu s'interprète grâce au cercle des corrélations : un athlète à droite du plan est du côté des fortes valeurs de `Points` et de saut en longueur, donc des temps de course faibles.
- Les **contributions** repèrent les individus qui pèsent le plus dans la construction d'un axe ; un individu au profil extrême peut à lui seul déterminer un axe.

5.1.3.4 Caractériser les axes avec les variables illustratives

La variable qualitative `Competition` est projetée au **barycentre** des individus de chaque modalité. La fonction `dimdesc()` décrit automatiquement chaque axe par les variables (actives et illustratives) qui lui sont le plus liées, avec un test de significativité :

```
description <- dimdesc(res_acp, axes = 1:2)
description$Dim.1$quanti      # corrélations des variables quantitatives avec
 ↪  l'axe 1
```

```
#>             correlation   p.value
#> Points           0.9562 2.099e-22
#> Long.jump        0.7419 2.850e-08
#> Shot.put         0.6225 1.388e-05
#> High.jump        0.5719 9.362e-05
#> Discus           0.5525 1.802e-04
#> Rank            -0.6705 1.616e-06
#> 400m            -0.6796 1.028e-06
#> 110m.hurdle     -0.7462 2.137e-08
#> 100m            -0.7747 2.778e-09
```

```
round(res_acp$quali.sup$v.test[, 1:2], 2)   # position des modalités, en
 ↪  valeur-test
```

```
#>          Dim.1 Dim.2
#> Decastar -1.43 -0.12
#> OlympicG  1.43  0.12
```

La **valeur-test** (*v.test*) mesure l'écart entre la position moyenne d'une modalité et le centre du nuage ; au-delà de 2 en valeur absolue, l'écart est significatif (au seuil de 5 % environ). Les athlètes des Jeux olympiques sont plutôt du côté des bonnes performances sur l'axe 1, mais l'écart n'est **pas significatif** (v.test d'environ 1,4) : avec 41 athlètes, on ne peut pas conclure que les performances diffèrent selon la compétition.

5.1.4 Exemple 2 : le comportement des clients de la boutique

Appliquons la même démarche aux clients de la boutique en ligne. La fonction `indicateurs_clients()` (chapitre 3) calcule pour chaque client actif en 2025 ses indicateurs de comportement :

Variable	Description
`recence`	jours depuis la dernière commande
`frequence`	nombre de commandes en 2025
`montant_total`	chiffre d'affaires 2025 (€)
`panier_moyen`	montant moyen d'une commande (€)
`part_promo`	part des commandes passées avec un code promo
`nb_categories`	nombre de catégories de produits différentes achetées
`articles_par_commande`	nombre moyen d'articles par commande

Variable	Description
`anciennete`	années depuis l'inscription

```
indicateurs <- indicateurs_clients(lire_boutique("commandes"),
 ↪  lire_boutique("clients"))

clients_acp <- indicateurs |>
  drop_na(age) |>                       # on écarte les clients sans âge
   ↪  renseigné
  select(client_id, recence, frequence, montant_total, panier_moyen,
   ↪  part_promo,
         nb_categories, articles_par_commande, anciennete,     # variables
 ↪  actives
         age, canal_acquisition, newsletter) |>                # variables
 ↪  illustratives
  column_to_rownames("client_id")
dim(clients_acp)
```

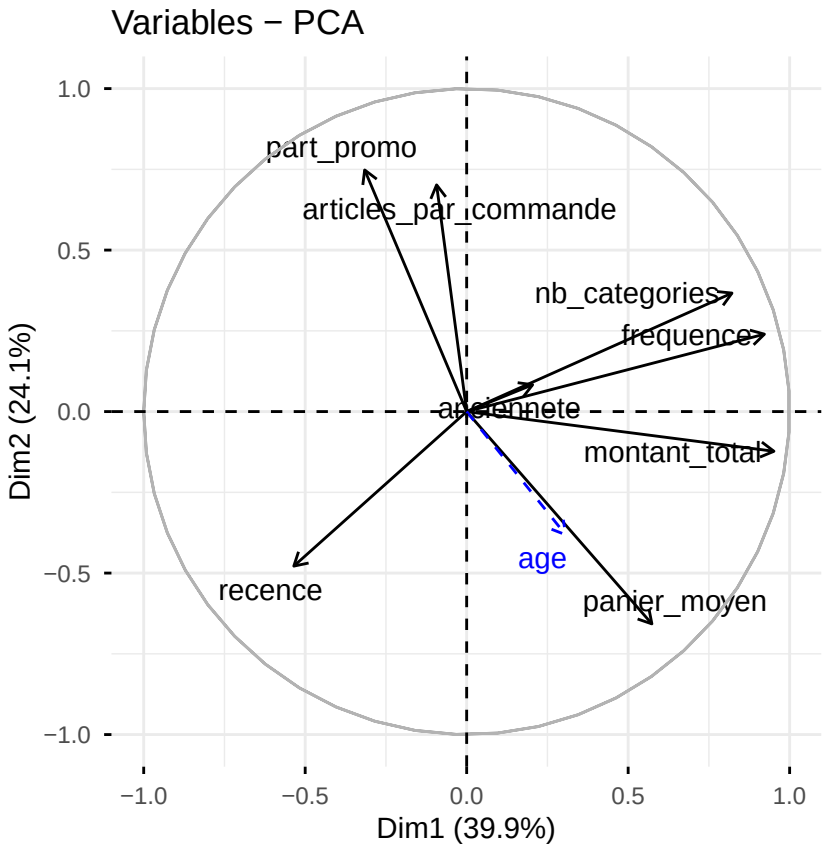
```
#> [1] 1688   11
```

Les huit indicateurs de comportement sont **actifs**. L'âge (quantitatif), le canal d'acquisition et l'abonnement à la newsletter (qualitatifs) sont **illustratifs** : on veut savoir s'ils sont liés aux profils de comportement, sans qu'ils construisent les axes.

```
acp_clients <- PCA(clients_acp, quanti.sup = 9, quali.sup = 10:11, graph =
 ↪  FALSE)
round(acp_clients$eig[1:4, ], 1)
```

```
#>        eigenvalue percentage of variance
#> comp 1        3.2                   39.9
#> comp 2        1.9                   24.1
#> comp 3        1.1                   13.3
#> comp 4        0.7                    8.3
#>        cumulative percentage of variance
#> comp 1                              39.9
#> comp 2                              64.0
#> comp 3                              77.3
#> comp 4                              85.6
```

```
fviz_pca_var(acp_clients, repel = TRUE)
```



Les deux premiers axes résument environ 64 % de l'inertie, et se lisent clairement :

- **Axe 1 — la valeur du client** : fréquence, montant total et nombre de catégories achetées, opposés à la récence (un client récent a une récence faible). À droite, les bons clients, actifs et fidèles ; à gauche, les clients peu actifs.
- **Axe 2 — le comportement promotionnel** : part des commandes en promotion et nombre d'articles par commande, opposés au panier moyen. En haut, des clients qui achètent beaucoup d'articles bon marché en promotion ; en bas, des clients aux paniers élevés, peu sensibles aux promotions.

L'ancienneté est mal représentée dans ce plan : elle porte à elle seule le troisième axe.

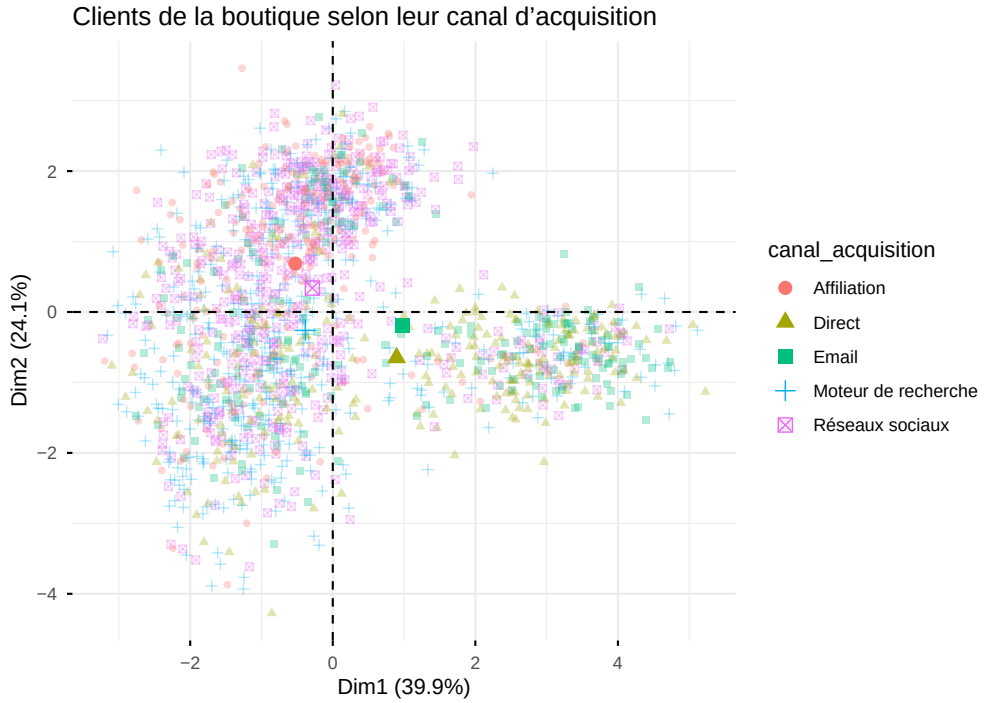
```
round(acp_clients$quanti.sup$coord[, 1:2], 2)     # âge
```

```
#> Dim.1 Dim.2
#>  0.31 -0.38
```

```
round(acp_clients$quali.sup$v.test[, 1:2], 1)      # canal et newsletter
```

```
#>                     Dim.1 Dim.2
#> Affiliation          -4.9   8.3
#> Direct                8.8  -8.2
#> Email                 8.7  -2.1
#> Moteur de recherche  -5.5  -4.8
#> Réseaux sociaux      -4.2   6.4
#> Non                  -9.0  -6.2
#> Oui                   9.0   6.2
```

```
fviz_pca_ind(acp_clients, geom = "point", habillage = "canal_acquisition",
             alpha.ind = 0.3, addEllipses = FALSE, mean.point = TRUE) +
  labs(title = "Clients de la boutique selon leur canal d'acquisition")
```



Les variables illustratives enrichissent l'interprétation :

- les clients venus en **direct** ou par **email** sont du côté des clients à forte valeur et peu sensibles aux promotions ;
- ceux venus par **affiliation** et **réseaux sociaux** sont du côté des promotions ;
- les **abonnés à la newsletter** sont plus engagés (axe 1) ;
- les clients **plus âgés** ont des paniers plus élevés et utilisent moins les promotions (corrélation négative avec l'axe 2).

Sur l'axe 1, toutes les valeurs-test des modalités dépassent 4 en valeur absolue : ces écarts ne sont pas dus au hasard.

Ces liens sont des **associations**. On ne peut pas en conclure que recruter par email « crée » de bons clients : les canaux attirent des publics différents. Rappelons aussi que ces données sont **simulées** ; sur des données réelles, les variables très asymétriques (montants) sont souvent transformées (logarithme) avant l'ACP, car quelques clients extrêmes peuvent sinon déterminer un axe à eux seuls.

Démarche d'une ACP

1. Choisir les variables actives (quantitatives) et illustratives.
2. ACP normée si les unités diffèrent.
3. Choisir le nombre d'axes (éboulis, Kaiser, interprétabilité).
4. Interpréter les axes avec le cercle des corrélations et les contributions.
5. Interpréter le plan des individus **à la lumière** du cercle.
6. Valider et enrichir l'interprétation avec les variables illustratives (`dimdesc()`, valeurs-test).
7. N'interpréter que les éléments bien représentés (`cos2`).

Pour explorer une ACP de façon interactive, le package `Factoshiny` propose une interface graphique : `Factoshiny::PCAshiny(decathlon)`.

5.2 Arbres de décision

5.2.1 Principe

Un arbre de décision prédit une variable cible en posant une **suite de questions binaires** sur les variables explicatives (« la fidélité du client à la marque est-elle supérieure à 0,48 ? »). C'est une méthode **supervisée** : on connaît la réponse sur les données d'apprentissage.

- **Classification** : la cible est qualitative (achète / n'achète pas) ; chaque feuille prédit la classe majoritaire.
- **Régression** : la cible est quantitative (montant) ; chaque feuille prédit la moyenne.

Construction (algorithme CART) : on part de tous les individus. À chaque nœud, on cherche la variable et le seuil qui séparent le mieux les individus en deux groupes **les plus homogènes possible** vis-à-vis de la cible. On répète dans chaque groupe (*partitionnement récursif*).

L'homogénéité d'un nœud se mesure par son **impureté**. Pour des proportions de classes $p_1, \dots, p_K$:

- indice de **Gini** : $G = 1 - \sum_k p_k^2$;
- **entropie** : $H = -\sum_k p_k \log p_k$.

Un nœud pur (une seule classe) a une impureté nulle. On choisit la coupure qui fait le plus baisser l'impureté.

Points forts : très lisible (on peut l'expliquer à une équipe commerciale), gère variables quantitatives et qualitatives, peu sensible aux échelles, détecte des interactions. **Points faibles** : instable (un petit changement des données peut changer tout l'arbre), tendance au **sur-apprentissage** si l'arbre est trop profond.

5.2.2 Les données : le choix d'une marque de jus d'orange

On reprend le jeu `OJ` (chapitre 4) : 1 070 achats de Citrus Hill (`CH`) ou de Minute Maid (`MM`), avec prix, remises, mises en avant et fidélité du client (`LoyalCH`). Objectif : prédire la marque achetée, par exemple pour **cibler** les clients à qui envoyer un bon de réduction MM.

On réserve un jeu de **test** dès le départ :

```
set.seed(2026)
indices_train <- sample(nrow(OJ), 800)
train <- OJ[indices_train, ]
test  <- OJ[-indices_train, ]
table(train$Purchase)
```

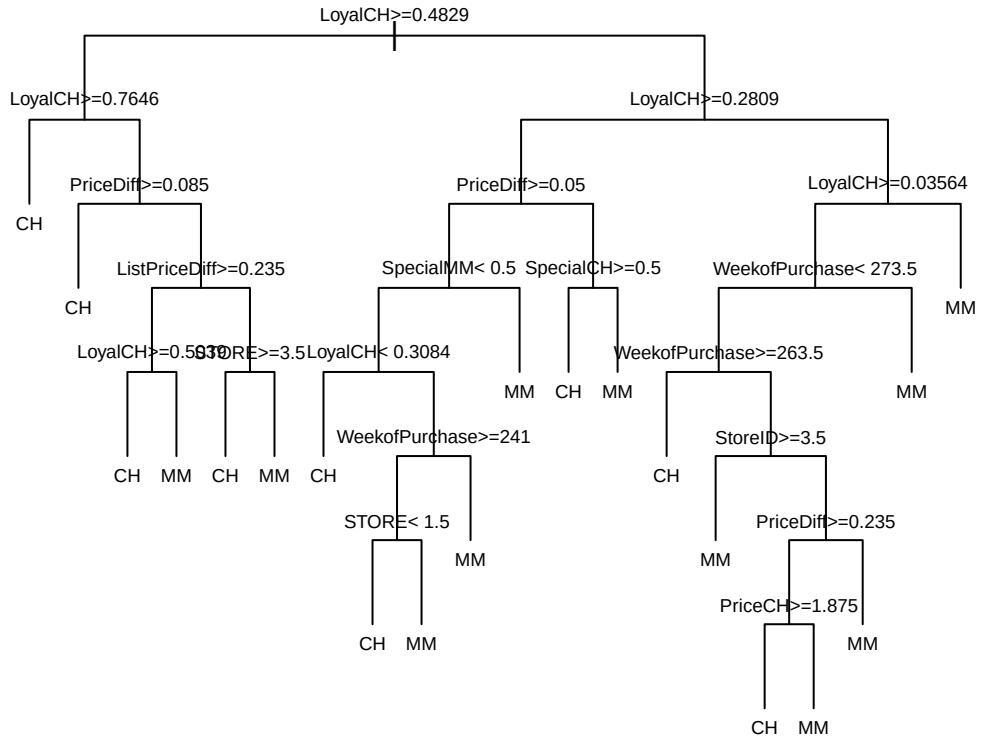
```
#>
#>  CH  MM
#> 485 315
```

5.2.3 Construire l'arbre

```
set.seed(2026)   # rpart utilise une validation croisée aléatoire
arbre <- rpart(Purchase ~ ., data = train, method = "class",
               control = rpart.control(cp = 0.001))
```

La formule `Purchase ~ .` signifie « `Purchase` expliqué par toutes les autres variables ». Avec un paramètre de complexité `cp` très faible, l'arbre est volontairement très détaillé :

```
plot(arbre, uniform = TRUE, margin = 0.05)
text(arbre, cex = 0.6)
```



Cet arbre apprend le bruit des données d'entraînement : il faut l'**élaguer**.

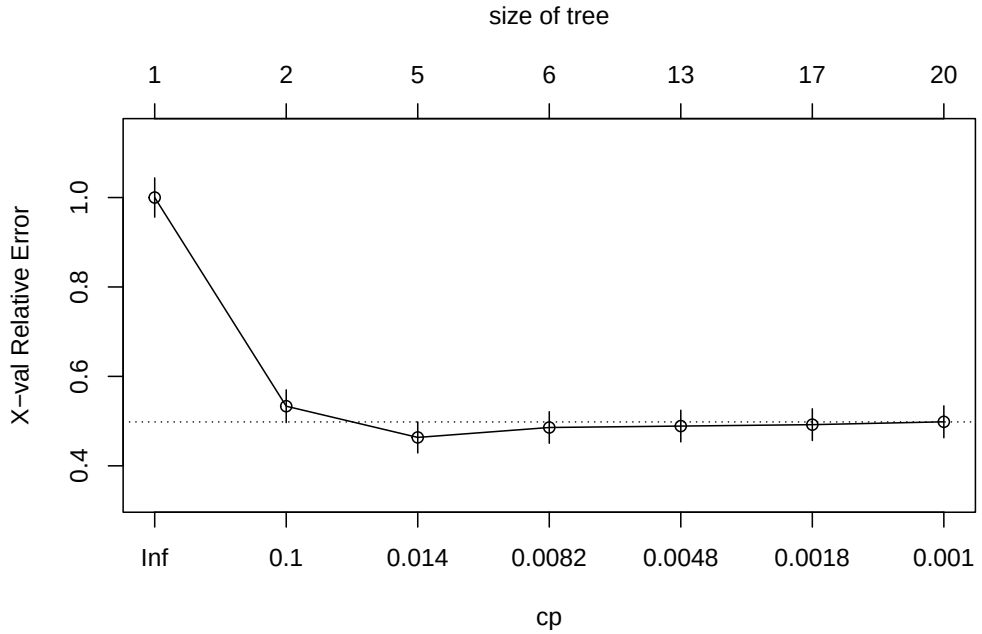
5.2.4 Élaguer l'arbre

Le **paramètre de complexité** `cp` pénalise les arbres trop grands. `rpart` estime l'erreur de chaque taille d'arbre par **validation croisée** (colonne `xerror`).

```
printcp(arbre)
```

```
#>
#> Classification tree:
#> rpart(formula = Purchase ~ ., data = train, method = "class",
#>     control = rpart.control(cp = 0.001))
#>
#> Variables actually used in tree construction:
#> [1] ListPriceDiff  LoyalCH        PriceCH        PriceDiff
#> [5] SpecialCH      SpecialMM      STORE          StoreID
#> [9] WeekofPurchase
#>
#> Root node error: 315/800 = 0.39
#>
#> n= 800
#>
#>       CP nsplit rel error xerror  xstd
#> 1 0.5175      0      1.00   1.00 0.044
#> 2 0.0201      1      0.48   0.53 0.037
#> 3 0.0095      4      0.42   0.46 0.035
#> 4 0.0071      5      0.41   0.49 0.035
#> 5 0.0032     12      0.36   0.49 0.035
#> 6 0.0011     16      0.34   0.49 0.035
#> 7 0.0010     19      0.34   0.50 0.036
```

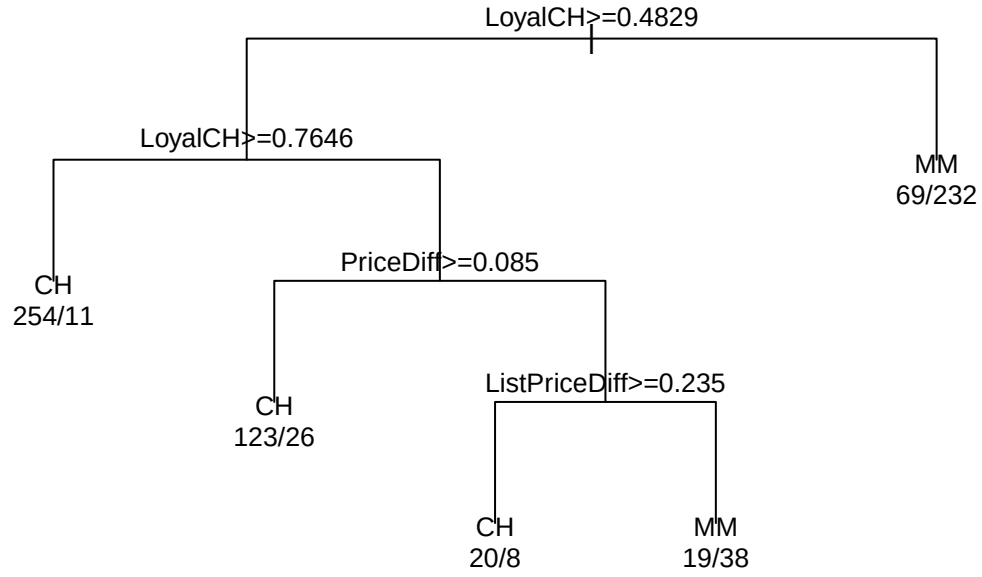
```
plotcp(arbre)
```



Règle usuelle : choisir le `cp` qui minimise `xerror`, ou le plus grand `cp` dont `xerror` reste à moins d'un écart-type du minimum (règle « 1-SE »), ce qui donne un arbre plus simple.

```
table_cp <- arbre$cptable
cp_optimal <- table_cp[which.min(table_cp[, "xerror"]), "CP"]
arbre_elague <- prune(arbre, cp = cp_optimal)

plot(arbre_elague, uniform = TRUE, margin = 0.08)
text(arbre_elague, use.n = TRUE, cex = 0.8, digits = 4)   # digits : affichage
 ↪  des effectifs
```



Lecture : chaque nœud indique une condition ; on va à **gauche** si elle est vraie. Chaque feuille affiche la classe prédite et les effectifs `CH/MM` d'entraînement. Le premier critère est la **fidélité** à Citrus Hill : les clients peu fidèles à CH achètent majoritairement MM. Pour les clients à fidélité moyenne, c'est la **différence de prix** qui fait basculer le choix. L'arbre se traduit directement

en règles commerciales.

```
round(arbre_elague$variable.importance, 1)
```

```
#>        LoyalCH    SalePriceMM      PriceDiff        StoreID
#>          154.8           14.9           14.6           12.8
#>        PriceMM  ListPriceDiff         DiscMM      PctDiscMM
#>          11.2           11.1           10.6           10.1
#> WeekofPurchase        PriceCH    SalePriceCH      SpecialCH
#>            5.6            4.7            1.9            1.4
#>         Store7
#>            1.2
```

5.2.5 Prédire

```
classe_predite <- predict(arbre_elague, newdata = test, type = "class")
proba_mm <- predict(arbre_elague, newdata = test, type = "prob")[, "MM"]
head(data.frame(observe = test$Purchase, predit = classe_predite,
                proba_mm = round(proba_mm, 2)))
```

```
#>    observe predit proba_mm
#> 1       CH     CH     0.17
#> 11      CH     CH     0.04
#> 13      CH     CH     0.04
#> 16      CH     CH     0.04
#> 22      CH     CH     0.17
#> 24      CH     CH     0.04
```

5.2.6 Matrice de confusion

On compare les classes prédites aux classes observées sur le jeu de **test**.

```
confusion <- table(observe = test$Purchase, predit = classe_predite)
confusion
```

```
#>        predit
#> observe  CH  MM
#>      CH 136  32
#>      MM  13  89
```

On appelle « positif » la classe d'intérêt, ici l'achat de `MM`.

	Prédit `CH` (négatif)	Prédit `MM` (positif)
Observé `CH`	vrais négatifs (VN)	faux positifs (FP)
Observé `MM`	faux négatifs (FN)	vrais positifs (VP)

Indicateur	Formule	Question
Taux de bonne classification (*accuracy*)	(VP + VN) / total	Quelle part est bien classée ?
Sensibilité (rappel)	VP / (VP + FN)	Parmi les acheteurs de MM, combien sont détectés ?
Spécificité	VN / (VN + FP)	Parmi les acheteurs de CH, combien sont reconnus ?

Indicateur	Formule	Question
Précision	VP / (VP + FP)	Parmi les clients ciblés, combien achètent vraiment MM ?

```r
evaluer <- function(confusion, positif) {
  negatif <- setdiff(colnames(confusion), positif)
  vp <- confusion[positif, positif]; fn <- confusion[positif, negatif]
  vn <- confusion[negatif, negatif]; fp <- confusion[negatif, positif]
  c(accuracy    = (vp + vn) / sum(confusion),
    sensibilite = vp / (vp + fn),
    specificite = vn / (vn + fp),
    precision   = vp / (vp + fp))
}
round(evaluer(confusion, positif = "MM"), 3)
```

```
#>    accuracy sensibilite specificite   precision
#>       0.833       0.873       0.810       0.736
```

En marketing, le choix de l'indicateur dépend du **coût des erreurs** : envoyer un bon de réduction à un client qui n'en fera rien (faux positif) coûte le prix de l'envoi ; ne pas cibler un client réceptif (faux négatif) coûte une vente perdue. Une campagne bon marché (email) privilégie la sensibilité ; une campagne coûteuse (courrier, remise importante) privilégie la précision.

> Le taux de bonne classification est trompeur quand les classes sont **déséquilibrées**. Si seulement 2 % des clients répondent à une campagne, un modèle qui prédit « personne ne répond » a 98 % de bonnes classifications… et ne sert à rien. L'exercice 5.2 en donne un exemple.

5.2.7 Courbe ROC et AUC

Un classifieur produit une **probabilité** ; la classe dépend d'un **seuil** (0,5 par défaut). Baisser le seuil augmente la sensibilité mais diminue la spécificité. La **courbe ROC** trace la sensibilité en fonction de 1 − spécificité pour **tous les seuils** possibles.

- Un classifieur parfait passe par le coin supérieur gauche.
- Un classifieur aléatoire suit la diagonale.
- L'**AUC** (aire sous la courbe) résume la courbe : 0,5 pour le hasard, 1 pour un classifieur parfait. C'est la probabilité qu'un individu positif tiré au hasard reçoive un score plus élevé qu'un individu négatif tiré au hasard.

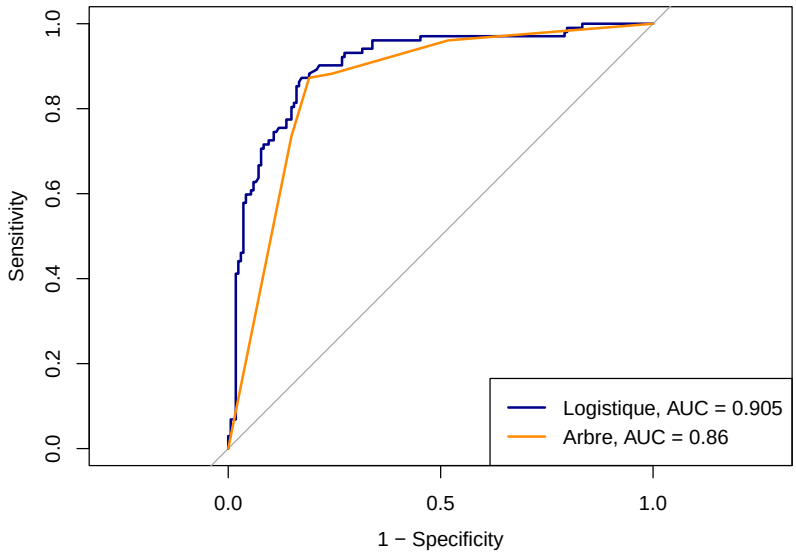
Comparons l'arbre à la régression logistique du chapitre 4 :

```r
logistique <- glm(Purchase ~ PriceDiff + LoyalCH + SpecialMM + SpecialCH,
                  data = train, family = binomial)
proba_log <- predict(logistique, newdata = test, type = "response")

roc_arbre <- roc(test$Purchase, proba_mm, levels = c("CH", "MM"), direction =
 ↪  "<")
roc_log   <- roc(test$Purchase, proba_log, levels = c("CH", "MM"), direction =
 ↪  "<")

plot(roc_log, col = "darkblue", legacy.axes = TRUE)
```

```r
plot(roc_arbre, col = "darkorange", add = TRUE)
legend("bottomright", col = c("darkblue", "darkorange"), lwd = 2,
       legend = c(paste("Logistique, AUC =", round(auc(roc_log), 3)),
                  paste("Arbre, AUC =", round(auc(roc_arbre), 3))))
```



La courbe de l'arbre est formée de segments : un arbre élagué n'a que quelques feuilles, donc quelques valeurs de probabilité possibles. Ici, la régression logistique discrimine un peu mieux ; l'arbre reste plus facile à expliquer. Ce compromis entre **performance** et **interprétabilité** est fréquent.

Choisir un seuil : la fonction `coords()` donne le seuil qui maximise sensibilité + spécificité (indice de Youden). En pratique, le seuil se choisit avec les équipes métier, en fonction du coût des erreurs et du budget de la campagne.

```r
coords(roc_log, "best", ret = c("threshold", "sensitivity", "specificity"))
```

```
#>   threshold sensitivity specificity
#> 1     0.389      0.8725      0.8274
```

> Les indicateurs doivent être calculés sur un jeu de **test** jamais utilisé pour construire ni régler le modèle. Mesurés sur le jeu d'entraînement, ils sont trop optimistes (voir la fuite de données au chapitre 1).

5.3 Forêts aléatoires

5.3.1 Pourquoi aller au-delà d'un arbre ?

Un arbre de décision est lisible, mais **instable** : un léger changement des données d'apprentissage peut modifier la première coupure, et donc tout l'arbre. En termes statistiques, ses prédictions ont une **variance** élevée. Un arbre profond a peu de biais (il s'adapte finement aux données) mais beaucoup de variance ; l'élagage réduit la variance au prix d'un peu de biais.

Les **méthodes d'ensemble** combinent de nombreux modèles pour réduire cette variance. La **forêt aléatoire** (*random forest*, Leo Breiman, 2001) est la plus utilisée d'entre elles : c'est l'une des méthodes les plus performantes « clés en main » sur des données tabulaires, comme on en rencontre sans cesse en marketing (clients, transactions, campagnes).

5.3.2 Rappels théoriques

5.3.2.1 Le bagging

Le *bagging* (*bootstrap aggregating*, Breiman, 1996) procède en trois étapes :

1. tirer B échantillons **bootstrap** : chacun contient n observations tirées **avec remise** parmi les n observations d'apprentissage ;
2. construire un arbre **profond, non élagué** sur chaque échantillon : $\hat{f}_1, \dots, \hat{f}_B$;
3. **agréger** les B prédictions.

Pourquoi cela marche-t-il ? Si l'on moyenne B prédictions de même variance σ^2 et de corrélation deux à deux ρ, la variance de la moyenne vaut

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B}\hat{f}_b(x)\right) = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2.$$

Le second terme disparaît quand B augmente : moyenner beaucoup d'arbres réduit la variance **sans augmenter le biais**. Mais le premier terme demeure : si les arbres se ressemblent trop (corrélation ρ forte), le gain est limité. Or, avec le bagging seul, les arbres se ressemblent : s'il existe une variable très prédictive (comme la fidélité `LoyalCH` du jeu `OJ`), presque tous les arbres la choisissent pour leur première coupure.

5.3.2.2 L'idée de la forêt aléatoire : décorréler les arbres

La forêt aléatoire ajoute une seconde source de hasard : **à chaque nœud** de chaque arbre, la meilleure coupure est cherchée parmi m variables **tirées au hasard** parmi les p variables explicatives (paramètre `mtry`), et non parmi toutes. Les variables moins dominantes ont ainsi leur chance, les arbres deviennent plus différents les uns des autres (ρ diminue), et la moyenne est plus efficace.

- Avec $m = p$, on retrouve le bagging.
- Valeurs par défaut de `randomForest` : $m = \lfloor\sqrt{p}\rfloor$ en classification, $m = \lfloor p/3\rfloor$ en régression.

5.3.2.3 Classification et régression

	Classification	Régression
Variable cible	qualitative (achète / n'achète pas)	quantitative (montant, ventes)
Critère de coupure dans chaque arbre	baisse de l'impureté de **Gini**	baisse de la **somme des carrés des résidus**
Taille minimale des feuilles (`nodesize`)	1 : arbres très profonds	5
`mtry` par défaut	$\lfloor\sqrt{p}\rfloor$	$\lfloor p/3\rfloor$
Prédiction de la forêt	**vote majoritaire** des B arbres ; la **probabilité** d'une classe est la part des arbres qui la prédisent	**moyenne** des prédictions des B arbres : $\hat{f}(x) = \frac{1}{B}\sum_b \hat{f}_b(x)$
Erreur mesurée	taux d'erreur, matrice de confusion, AUC	erreur quadratique moyenne (MSE, RMSE), part de variance expliquée

En **régression**, une forêt prédit une moyenne de valeurs observées dans les feuilles : elle **ne peut pas extrapoler**. Ses prédictions restent dans l'intervalle des valeurs de la cible vues à l'apprentissage, et elle prolonge mal une tendance (par exemple des ventes en croissance continue).

5.3.2.4 L'erreur *out-of-bag*

Un échantillon bootstrap ne contient pas toutes les observations : la probabilité qu'une observation donnée n'y figure pas est $(1 - 1/n)^n \approx e^{-1} \approx 0{,}37$. Environ un tiers des observations sont donc « hors du sac » (*out-of-bag*, OOB) pour chaque arbre.

Pour chaque observation, on peut agréger les prédictions des seuls arbres qui **ne l'ont pas vue** : on obtient l'**erreur OOB**, une estimation de l'erreur de prédiction comparable à une validation croisée, **sans jeu de validation séparé**. Elle sert notamment à régler `mtry`. Elle ne dispense pas de garder un jeu de **test** pour l'évaluation finale si l'on compare plusieurs méthodes.

5.3.2.5 Importance des variables

Une forêt de 500 arbres ne se lit pas comme un arbre unique. Deux mesures résument le rôle de chaque variable :

- **diminution moyenne de l'impureté** (`MeanDecreaseGini` en classification, `IncNodePurity` en régression) : total des baisses d'impureté obtenues grâce aux coupures sur cette variable, moyenné sur les arbres. Rapide à calculer, mais elle favorise les variables quantitatives ou à nombreuses modalités ;
- **importance par permutation** (`MeanDecreaseAccuracy` en classification, `%IncMSE` en régression) : on **mélange au hasard** les valeurs d'une variable dans les observations OOB et l'on mesure la **dégradation** de la prédiction. Si mélanger une variable ne change rien, elle n'est pas utile au modèle. Plus fiable, elle demande `importance = TRUE`.

L'importance mesure l'**utilité pour prédire**, pas un effet causal, et ne donne pas le **sens** de l'effet. Deux variables très corrélées se partagent l'importance : chacune peut paraître peu importante alors que l'information qu'elles portent l'est.

5.3.2.6 Paramètres à régler

Paramètre	Rôle	Conseil
`ntree`	nombre d'arbres B	500 par défaut ; augmenter ne provoque **pas** de sur-apprentissage, seulement un calcul plus long : vérifier que l'erreur OOB s'est stabilisée (`plot()`)
`mtry`	variables candidates à chaque nœud	le paramètre principal ; à régler avec l'erreur OOB
`nodesize`, `maxnodes`	taille des feuilles, nombre de feuilles	profondeur des arbres ; les valeurs par défaut conviennent souvent
`sampsize`, `strata`, `classwt`	taille et composition des échantillons bootstrap	utiles quand les classes sont **déséquilibrées** (exercice 5.5)

Points forts : très bonnes performances avec peu de réglages, variables quantitatives et qualitatives sans transformation, interactions et effets non linéaires détectés automatiquement, estimation OOB gratuite. **Points faibles** : modèle peu interprétable (une « boîte noire » qu'on explore avec l'importance des variables et les graphiques de dépendance partielle), calcul plus lourd qu'un arbre, pas d'extrapolation en régression, pas de valeurs manquantes acceptées par `randomForest` (les traiter avant, par exemple avec `na.roughfix()`). Pour de grands volumes de données, le package `ranger` est beaucoup plus rapide.

5.3.3 Classification : le choix de la marque de jus d'orange

On reprend les données `OJ` et le **même découpage** entraînement / test que pour l'arbre (`train` : 800 achats, `test` : 270 achats), afin de comparer les méthodes sur les mêmes observations.

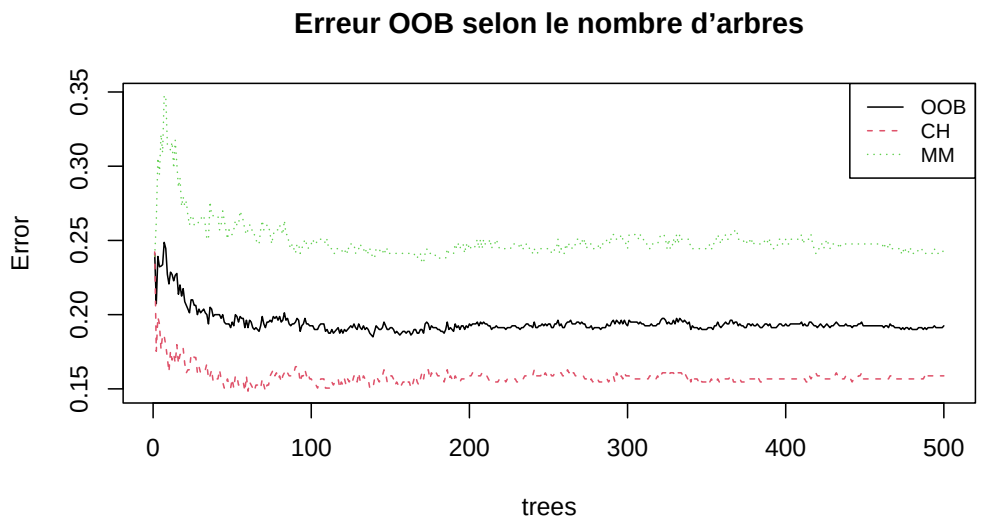
```
set.seed(2026)
foret <- randomForest(Purchase ~ ., data = train, ntree = 500, importance =
 ↪  TRUE)
foret
```

```
#>
#> Call:
#>  randomForest(formula = Purchase ~ ., data = train, ntree = 500,      importance =
 ↪  TRUE)
#>                Type of random forest: classification
#>                      Number of trees: 500
#> No. of variables tried at each split: 4
#>
#>         OOB estimate of  error rate: 19.25%
#> Confusion matrix:
#>     CH  MM class.error
#> CH 408  77      0.1588
#> MM  77 238      0.2444
```

La sortie indique le type de forêt (classification), le nombre d'arbres, la valeur de `mtry` (ici $\lfloor\sqrt{17}\rfloor = 4$), l'**erreur OOB** et la matrice de confusion OOB, calculées sur les données d'apprentissage sans jeu de validation.

L'erreur s'est-elle stabilisée ? `plot()` trace l'erreur OOB (et celle de chaque classe) en fonction du nombre d'arbres :

```
plot(foret, main = "Erreur OOB selon le nombre d'arbres")
legend("topright", colnames(foret$err.rate), col = 1:3, lty = 1:3, cex = 0.8)
```



Au-delà de 150 à 200 arbres, l'erreur ne fait plus qu'osciller légèrement : 500 arbres suffisent largement.

Régler `mtry` avec l'erreur OOB :

```
erreur_oob <- sapply(1:10, function(m) {
  set.seed(2026)
  modele <- randomForest(Purchase ~ ., data = train, mtry = m)
  modele$err.rate[modele$ntree, "OOB"]
})
round(setNames(erreur_oob, 1:10), 3)
```

```
#>     1     2     3     4     5     6     7     8     9    10
#> 0.229 0.202 0.189 0.189 0.198 0.194 0.199 0.201 0.208 0.202
```

L'erreur est la plus faible autour de $m = 3$ ou 4 ; la valeur par défaut convient. Avec $m = 1$, les coupures sont presque aléatoires ; avec m grand, on se rapproche du bagging.

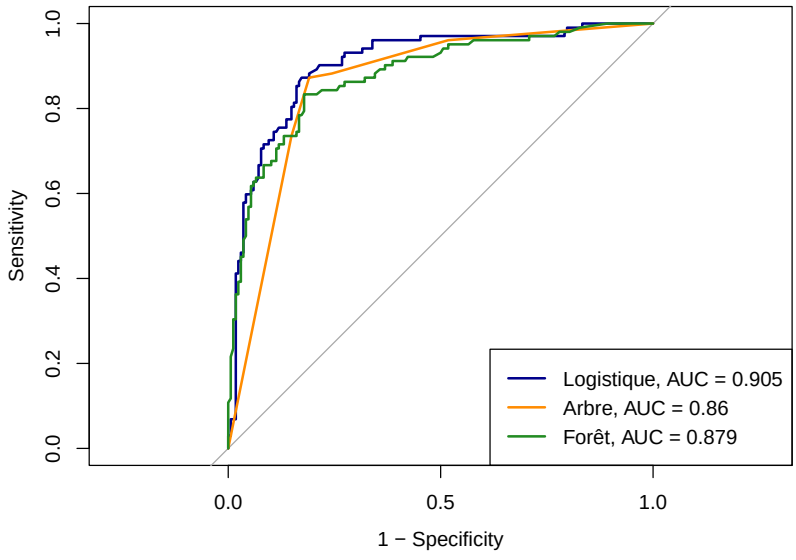
Évaluer sur le jeu de test et comparer à l'arbre élagué et à la régression logistique :

```
table(observe = test$Purchase, predit = predict(foret, newdata = test))
```

```
#>        predit
#> observe  CH  MM
#>      CH 138  30
#>      MM  20  82
```

```
proba_foret <- predict(foret, newdata = test, type = "prob")[, "MM"]
roc_foret <- roc(test$Purchase, proba_foret, levels = c("CH", "MM"), direction
 ↪  = "<")

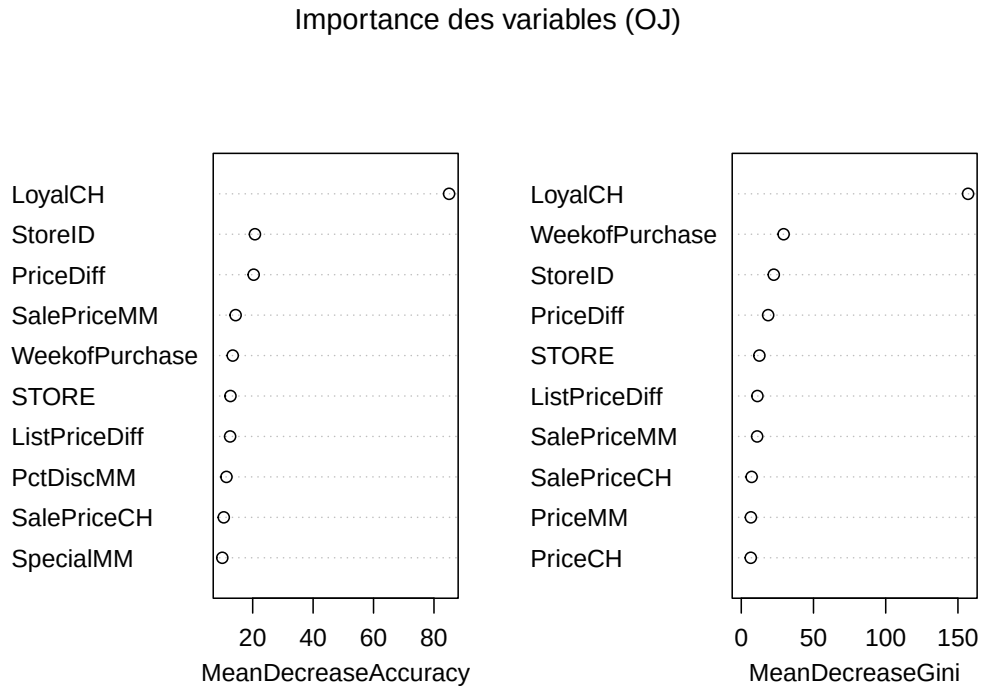
plot(roc_log, col = "darkblue", legacy.axes = TRUE)
plot(roc_arbre, col = "darkorange", add = TRUE)
plot(roc_foret, col = "forestgreen", add = TRUE)
legend("bottomright", col = c("darkblue", "darkorange", "forestgreen"), lwd =
 ↪  2,
       legend = c(paste("Logistique, AUC =", round(auc(roc_log), 3)),
                  paste("Arbre, AUC =", round(auc(roc_arbre), 3)),
                  paste("Forêt, AUC =", round(auc(roc_foret), 3))))
```



La forêt fait mieux que l'arbre unique : agréger 500 arbres stabilise les prédictions et donne des probabilités plus fines. Elle ne dépasse pourtant pas la régression logistique : dans ces données, l'essentiel de l'information tient dans deux variables (`LoyalCH` et `PriceDiff`) dont l'effet est presque linéaire sur l'échelle du logit, situation idéale pour un modèle logistique.

Importance des variables :

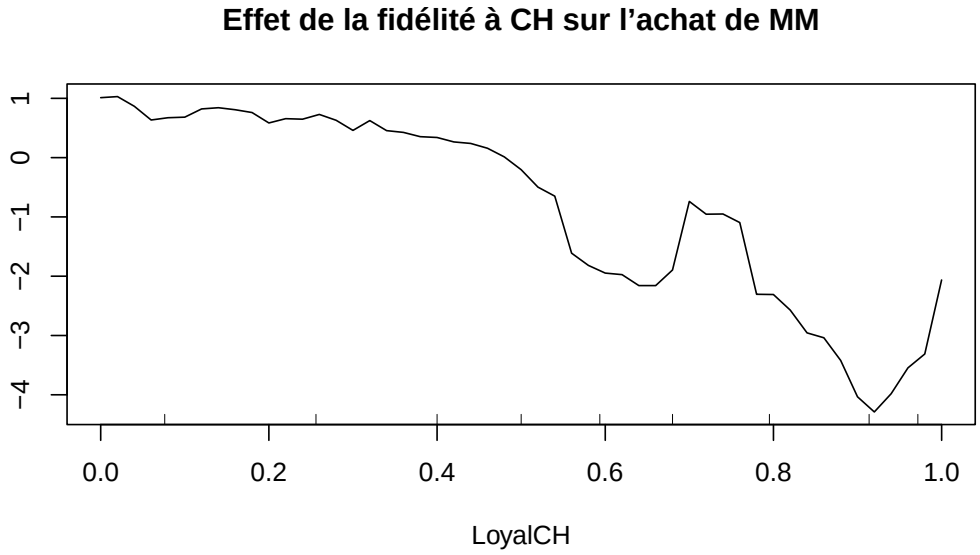
```
varImpPlot(foret, n.var = 10, main = "Importance des variables (OJ)")
```



La fidélité à la marque domine nettement, devant le magasin et les écarts de prix : on retrouve, sous une autre forme, les conclusions de l'arbre et de la régression logistique.

Dépendance partielle. Pour voir **comment** une variable influence la prédiction, `partialPlot()` trace la prédiction moyenne de la forêt quand on fait varier cette variable, les autres restant à leurs valeurs observées (l'axe vertical est sur l'échelle du logit de la probabilité de la classe choisie) :

```
partialPlot(foret, pred.data = as.data.frame(train), x.var = "LoyalCH",
            which.class = "MM", main = "Effet de la fidélité à CH sur l'achat
             ↪  de MM",
            xlab = "LoyalCH")
```



Plus un client est fidèle à Citrus Hill, moins la forêt prédit l'achat de Minute Maid, avec une chute marquée autour de 0,5. Les irrégularités de la courbe (bosse vers 0,7, remontée près de 1) ne doivent pas être surinterprétées : la forêt est une moyenne de fonctions en escalier, et certaines valeurs de `LoyalCH` sont rares dans les données. On lit la **tendance générale**, pas chaque variation.

5.3.4 Régression : les ventes de sièges auto

On reprend le jeu `Carseats` (chapitre 4) pour prédire les ventes `Sales`, avec le même découpage 70 % / 30 % qu'au chapitre 4.

```
set.seed(2026)
indices_train <- sample(nrow(Carseats), size = round(0.7 * nrow(Carseats)))
ventes_train <- Carseats[indices_train, ]
ventes_test  <- Carseats[-indices_train, ]

set.seed(2026)
foret_ventes <- randomForest(Sales ~ ., data = ventes_train, importance = TRUE)
foret_ventes
```

```
#>
#> Call:
#>  randomForest(formula = Sales ~ ., data = ventes_train, importance = TRUE)
#>                Type of random forest: regression
#>                      Number of trees: 500
#> No. of variables tried at each split: 3
#>
#>           Mean of squared residuals: 2.673
#>                     % Var explained: 64.83
```

En régression, la sortie donne l'erreur quadratique moyenne OOB (*Mean of squared residuals*) et la part de variance expliquée, l'équivalent d'un R^2 calculé hors échantillon. Ici, `mtry` vaut $\lfloor 10/3 \rfloor = 3$.

Comparons sur le jeu de test la référence naïve (la moyenne), un arbre de régression, la forêt et le modèle linéaire multiple du chapitre 4 :

```
rmse <- function(observe, predit) sqrt(mean((observe - predit)^2))
arbre_ventes <- rpart(Sales ~ ., data = ventes_train)
```

```r
lineaire <- lm(Sales ~ Price + CompPrice + Advertising + ShelveLoc + Age +
 ↪  Income,
               data = ventes_train)
c(reference = rmse(ventes_test$Sales, mean(ventes_train$Sales)),
  arbre     = rmse(ventes_test$Sales, predict(arbre_ventes, ventes_test)),
  foret     = rmse(ventes_test$Sales, predict(foret_ventes, ventes_test)),
  lineaire  = rmse(ventes_test$Sales, predict(lineaire, ventes_test)))
```

```
#> reference     arbre     foret  lineaire
#>     2.971     2.508     1.809     1.059
```

La forêt améliore nettement l'arbre unique, mais le **modèle linéaire** reste le meilleur. Les ventes de `Carseats` sont en effet construites comme une fonction presque **linéaire** du prix, de l'emplacement, de la publicité… Un modèle linéaire capture exactement cette structure ; une forêt doit l'approcher par une moyenne de fonctions en escalier.

```r
varImpPlot(foret_ventes, main = "Importance des variables (Carseats)")
```



L'emplacement en rayon et le prix dominent, comme dans le modèle linéaire. Les variables que le modèle linéaire n'avait pas retenues (population, niveau d'études, zone urbaine) ont une importance **par permutation** (`%IncMSE`) quasi nulle ; leur diminution d'impureté (`IncNodePurity`) n'est pourtant pas négligeable, car cette mesure favorise les variables quantitatives, sur lesquelles les arbres trouvent toujours des coupures, même inutiles.

> **Une forêt aléatoire est un excellent point de départ, pas une garantie.** Comparez-la toujours, sur des données de test, à une référence naïve et à un modèle simple et interprétable (régression linéaire ou logistique). Si le modèle simple fait aussi bien, préférez-le : il s'explique plus facilement aux équipes métier et aux personnes concernées par la décision.

5.4 Résumé

- **ACP** : résumer p variables quantitatives par quelques axes non corrélés qui conservent le maximum d'inertie. ACP normée si les unités diffèrent.
- `PCA(données, quanti.sup = ..., quali.sup = ..., graph = FALSE)` ; `fviz_eig()`, `fviz_pca_var()`, `fviz_pca_ind()`, `dimdesc()`.
- Cercle des corrélations : angle entre flèches = corrélation ; n'interpréter que les variables bien représentées.
- Les variables **illustratives** ne construisent pas les axes ; elles aident à les interpréter (valeurs-test).
- **Arbre de décision** : partitionnement récursif minimisant l'impureté (Gini) ; `rpart()`, `printcp()`, `prune()`, `predict()`.
- Évaluer sur un jeu de **test** : matrice de confusion, sensibilité, spécificité, précision ; courbe ROC et AUC avec `pROC`. Choisir le seuil selon le coût des erreurs.
- **Forêt aléatoire** : moyenne de nombreux arbres profonds construits sur des échantillons bootstrap, avec m variables tirées au hasard à chaque nœud (`mtry`) pour décorréler les arbres. Vote majoritaire en classification, moyenne en régression. `randomForest()`, erreur OOB, `varImpPlot()`, `partialPlot()`. Toujours comparer à un modèle simple.

5.5 Exercices

Exercice 5.1 — ACP sur `USArrests`. Le jeu `USArrests` donne, pour les 50 États américains, le nombre d'arrestations pour 100 000 habitants pour meurtre, agression et viol, ainsi que le pourcentage de population urbaine.

1. Faut-il réaliser une ACP normée ? Justifier.
2. Réaliser l'ACP. Combien d'axes retenir ?
3. Interpréter les deux premiers axes à l'aide du cercle des corrélations.
4. Quels États ont les profils les plus extrêmes ?

Solution

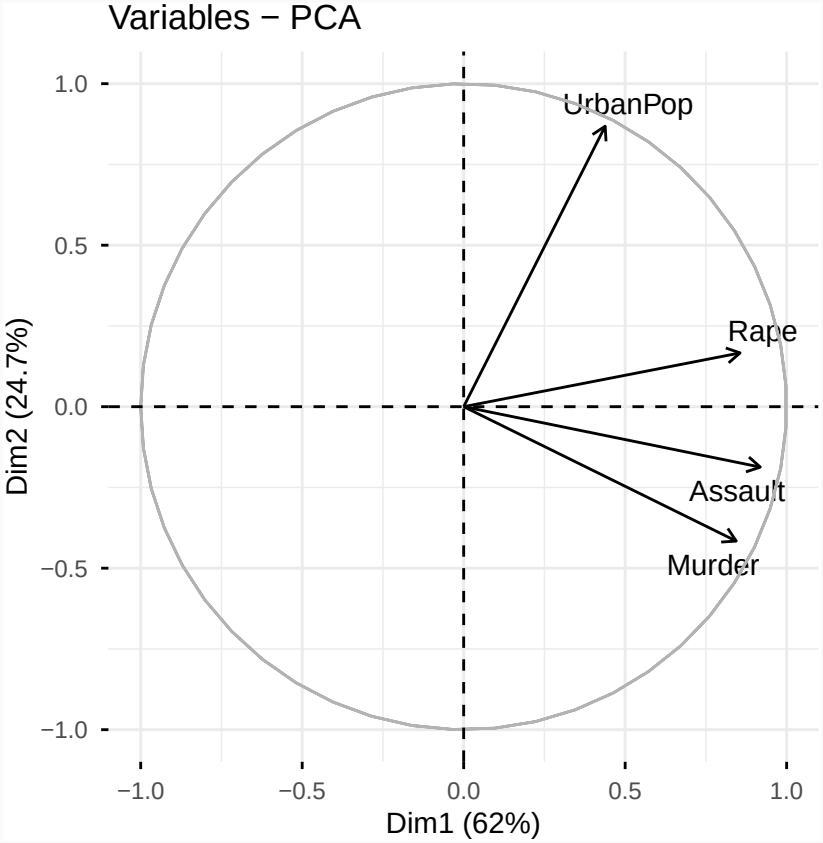
```
sapply(USArrests, var)        # variances très différentes : ACP normée
 ↪  nécessaire
```

```
#>   Murder  Assault UrbanPop     Rape
#>    18.97  6945.17   209.52    87.73
```

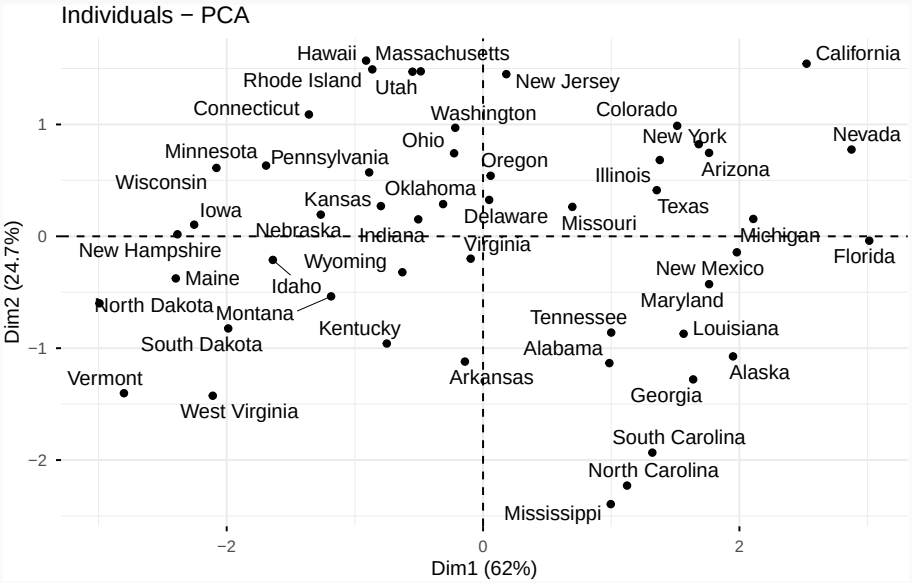
```
acp_us <- PCA(USArrests, graph = FALSE)
acp_us$eig
```

```
#>        eigenvalue percentage of variance
#> comp 1     2.4802                 62.006
#> comp 2     0.9898                 24.744
#> comp 3     0.3566                  8.914
#> comp 4     0.1734                  4.336
#>        cumulative percentage of variance
#> comp 1                             62.01
#> comp 2                             86.75
#> comp 3                             95.66
#> comp 4                            100.00
```

```
fviz_pca_var(acp_us, repel = TRUE)
```




```
fviz_pca_ind(acp_us, repel = TRUE)
```



L'axe 1 (environ 62 % de l'inertie) regroupe les trois types de crimes : c'est un axe de **criminalité globale**. L'axe 2 est porté par `UrbanPop` : il oppose les États **urbains** aux États **ruraux**.

Exercice 5.2 — Ciblage d'une campagne d'assurance. Le jeu `Caravan` (package `ISLR`) contient les données **réelles** de 5 822 clients d'un assureur néerlandais, diffusées lors du concours *CoIL Challenge 2000*. L'assureur voulait savoir quels clients cibler pour vendre

une assurance pour caravane (camping-car). Chaque client est décrit par 86 variables :

- 43 variables **socio-démographiques** (noms commençant par `M`, par exemple `MOSTYPE`, le type de client, ou `MKOOPKLA`, la classe de pouvoir d'achat). Elles ne décrivent pas le client lui-même mais **son quartier** : tous les clients d'un même code postal ont les mêmes valeurs. La plupart sont codées en classes de 0 à 9 (par exemple, part des habitants du quartier ayant tel revenu) ;
- 42 variables de **détention de produits** d'assurance : niveau des cotisations par type de contrat, codé en classes (noms commençant par `P`, par exemple `PPERSAUT` pour l'assurance automobile) et nombre de contrats (noms commençant par `A`, par exemple `APERSAUT`) ;
- la cible `Purchase` : le client a-t-il souscrit l'assurance caravane (`Yes` / `No`) ?

Les variables de quartier illustrent une pratique courante en marketing (le **géomarketing**) et ses limites : on attribue à chaque client les caractéristiques moyennes de son voisinage.

1. Quelle est la proportion de souscripteurs ? Que vaut le taux de bonne classification d'un modèle qui prédit toujours « No » ?
2. Découper les données (70 % / 30 %) et construire un arbre de classification. Que prédit-il sur le jeu de test ?
3. Utiliser plutôt les **probabilités** prédites : tracer la courbe ROC et calculer l'AUC.
4. L'assureur peut contacter 10 % des clients. En ciblant les 10 % de clients du jeu de test aux probabilités les plus élevées, quelle part des souscripteurs atteint-il ? Comparer au hasard (10 %).

Solution

```
mean(Caravan$Purchase == "Yes")          # environ 6 % : prédire "No"
 ↪  donne 94 % de bonnes réponses
```

```
#> [1] 0.05977
```

```
set.seed(1)
idx <- sample(nrow(Caravan), round(0.7 * nrow(Caravan)))
car_train <- Caravan[idx, ]
car_test  <- Caravan[-idx, ]

arbre_car <- rpart(Purchase ~ ., data = car_train, method = "class",
                   control = rpart.control(cp = 0.002))
table(observe = car_test$Purchase,
      predit = predict(arbre_car, car_test, type = "class"))
```

```
#>        predit
#> observe   No  Yes
#>     No  1629   13
#>     Yes  101    4
```

```
logit_car <- glm(Purchase ~ ., data = car_train, family = binomial)
```

```
#> Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
score <- predict(logit_car, car_test, type = "response")
auc(roc(car_test$Purchase, score, levels = c("No", "Yes"), direction =
 ↪  "<"))
```

```
#> Area under the curve: 0.729
```

```r
cibles <- score >= quantile(score, 0.9)          # les 10 % de meilleurs
↪ scores
sum(car_test$Purchase[cibles] == "Yes") / sum(car_test$Purchase == "Yes")
```

```
#> [1] 0.3238
```

Avec le seuil de 0,5, l'arbre ne prédit presque jamais « Yes » : son taux de bonne classification est élevé mais il ne sert à rien pour cibler. En classant les clients par probabilité et en ne contactant que les 10 % les mieux notés, on atteint une part des souscripteurs bien supérieure à 10 % : c'est ce gain (*lift*) qui justifie un modèle de ciblage. La régression logistique sur 85 variables peut produire des avertissements de convergence ; les modèles régularisés (hors programme) sont plus adaptés.

Exercice 5.3 — Questions sur les forêts aléatoires.

1. Pourquoi agréger de nombreux arbres réduit-il l'erreur de prédiction ? Quelle composante de l'erreur (biais ou variance) est réduite ?
2. Quelle est la différence entre le bagging et une forêt aléatoire ? Pourquoi cette différence améliore-t-elle les performances ?
3. Comment une forêt produit-elle sa prédiction en classification ? En régression ?
4. Quelle part des observations est « hors du sac » pour un arbre donné ? À quoi sert l'erreur OOB ?
5. Augmenter le nombre d'arbres peut-il conduire au sur-apprentissage ?
6. Une équipe marketing conclut d'une forte importance de la variable « nombre de visites du site » qu'augmenter les visites fera augmenter les achats. Qu'en pensez-vous ?

Éléments de réponse

1. Les arbres profonds ont un faible biais mais une forte **variance** : ils dépendent beaucoup de l'échantillon d'apprentissage. La moyenne de B prédictions a une variance $\rho\sigma^2 + (1-\rho)\sigma^2/B$, plus faible que celle d'un arbre, sans augmentation du biais.
2. Le bagging construit chaque arbre sur un échantillon bootstrap en considérant toutes les variables à chaque nœud ; la forêt aléatoire ne considère que m variables tirées au hasard. Les arbres sont alors moins corrélés (ρ plus faible), donc leur moyenne réduit davantage la variance.
3. Classification : vote majoritaire des arbres (et part des votes comme probabilité). Régression : moyenne des prédictions des arbres.
4. Environ $e^{-1} \approx 37\,\%$. L'erreur OOB, calculée en prédisant chaque observation avec les seuls arbres qui ne l'ont pas utilisée, estime l'erreur de prédiction sans jeu de validation ; elle sert à choisir `mtry` et à vérifier que le nombre d'arbres suffit.
5. Non : ajouter des arbres stabilise la moyenne, l'erreur se stabilise puis ne bouge plus. Seul le temps de calcul augmente.
6. L'importance mesure l'utilité d'une variable pour **prédire**, pas un effet **causal** : les clients qui visitent souvent le site sont déjà intéressés. Pour mesurer l'effet d'une action (augmenter les visites), il faut une expérience, par exemple un test A/B.

Exercice 5.4 — Forêt aléatoire en régression. Avec `phbirths` (`datasets/phbirths.dat` : poids de naissance `grams`, durée de gestation `gestate`, tabagisme de la mère `smoke`, origine `black`, années d'études `educ`) :

1. Découper les données en 70 % pour l'entraînement et 30 % pour le test.
2. Ajuster une forêt aléatoire de `grams` sur toutes les autres variables. Que vaut `mtry` ?

Quelle part de variance est expliquée (OOB) ?
3. Choisir `mtry` (de 1 à 4) avec l'erreur OOB.
4. Comparer sur le jeu de test le RMSE de la forêt retenue à celui d'une régression linéaire et à celui de la référence naïve (la moyenne). Conclure.

Solution

```
phbirths <- read.table("../datasets/phbirths.dat")
set.seed(1)
idx <- sample(nrow(phbirths), round(0.7 * nrow(phbirths)))
ph_train <- phbirths[idx, ]
ph_test  <- phbirths[-idx, ]

set.seed(1)
foret_ph <- randomForest(grams ~ ., data = ph_train)
foret_ph                                        # mtry = floor(4 / 3) = 1
```

```
#>
#> Call:
#>  randomForest(formula = grams ~ ., data = ph_train)
#>                Type of random forest: regression
#>                      Number of trees: 500
#> No. of variables tried at each split: 1
#>
#>           Mean of squared residuals: 222885
#>                     % Var explained: 42.7
```

```
erreur_oob <- sapply(1:4, function(m) {
  set.seed(1)
  tail(randomForest(grams ~ ., data = ph_train, mtry = m)$mse, 1)
})
round(setNames(erreur_oob, 1:4))
```

```
#>      1      2      3      4
#> 222885 192152 199950 205937
```

```
set.seed(1)
foret_ph <- randomForest(grams ~ ., data = ph_train, mtry =
↪  which.min(erreur_oob))
lineaire_ph <- lm(grams ~ ., data = ph_train)
c(reference = rmse(ph_test$grams, mean(ph_train$grams)),
  lineaire  = rmse(ph_test$grams, predict(lineaire_ph, ph_test)),
  foret     = rmse(ph_test$grams, predict(foret_ph, ph_test)))
```

```
#> reference  lineaire     foret
#>     655.4     431.7     452.1
```

Le réglage de `mtry` améliore la forêt, mais la régression linéaire reste meilleure. Avec seulement quatre variables explicatives et une relation essentiellement linéaire, une forêt n'a pas d'avantage : la régression linéaire, plus simple et interprétable, est à préférer.

Exercice 5.5 — Forêt aléatoire et classes déséquilibrées. On reprend le jeu `Caravan` et le découpage de l'exercice 5.2 (`car_train`, `car_test`).

1. Ajuster une forêt aléatoire de `Purchase` sur toutes les variables. Examiner la matrice de confusion OOB : combien de souscripteurs sont détectés ?
2. Calculer l'AUC sur le jeu de test et la part des souscripteurs atteinte en ciblant les 10

% de clients aux probabilités les plus élevées. Comparer à la régression logistique de l'exercice 5.2.

3. Refaire la forêt en construisant chaque arbre sur un échantillon **équilibré** de 200 non-souscripteurs et 200 souscripteurs (`sampsize = c(No = 200, Yes = 200), strata = car_train$Purchase`). Comparer la matrice de confusion sur le jeu de test et l'AUC.

Solution

```
set.seed(1)
foret_car <- randomForest(Purchase ~ ., data = car_train)
foret_car$confusion                          # presque aucun souscripteur
 ↪  détecté
```

```
#>       No Yes class.error
#> No  3798  34    0.008873
#> Yes  242   1    0.995885
```

```
score_foret <- predict(foret_car, car_test, type = "prob")[, "Yes"]
auc(roc(car_test$Purchase, score_foret, levels = c("No", "Yes"), direction
 ↪  = "<"))
```

```
#> Area under the curve: 0.688
```

```
cibles <- score_foret >= quantile(score_foret, 0.9)
sum(car_test$Purchase[cibles] == "Yes") / sum(car_test$Purchase == "Yes")
```

```
#> [1] 0.2952
```

```
set.seed(1)
foret_equilibree <- randomForest(Purchase ~ ., data = car_train,
                                 sampsize = c(No = 200, Yes = 200),
                                 strata = car_train$Purchase)
table(observe = car_test$Purchase, predit = predict(foret_equilibree,
 ↪  car_test))
```

```
#>        predit
#> observe   No  Yes
#>     No  1370  272
#>     Yes   56   49
```

```
score_eq <- predict(foret_equilibree, car_test, type = "prob")[, "Yes"]
auc(roc(car_test$Purchase, score_eq, levels = c("No", "Yes"), direction =
 ↪  "<"))
```

```
#> Area under the curve: 0.719
```

Avec les réglages par défaut, chaque arbre voit très peu de souscripteurs (environ 6 %) et vote presque toujours « No » : la forêt détecte à peine un souscripteur, et son classement est moins bon que celui de la régression logistique. En équilibrant les échantillons bootstrap, chaque arbre voit autant de souscripteurs que de non-souscripteurs : la forêt en détecte alors une bonne partie, au prix de nombreux faux positifs, et son AUC s'améliore. Le choix entre ces deux comportements dépend, encore une fois, du coût d'un contact inutile face à celui d'une vente manquée.

Pour aller plus loin

- F. Husson, S. Lê et J. Pagès, *Analyse de données avec R*, chapitre ACP ; vidéos de F. Husson sur YouTube (chaîne « François Husson »), en français.

- G. Saporta, *Probabilités, analyse des données et statistique*, chapitre 7.
- G. James et al., *An Introduction to Statistical Learning*, chapitres 8 (arbres, bagging et forêts aléatoires) et 12 (ACP).
- L. Breiman, « Random Forests », *Machine Learning*, 45, 2001, p. 5-32.